APPLICATION WEB DJANGO : GESTION DES ETUDIANTS

Architecture MVT, Modèles, Administration et Authentification

Introduction

Ce TP guidé est la suite logique du TP MI-guidé portant sur l'installation et l'architecture de base de Django. Dans ce deuxième TP, vous allez construire une application web complète de gestion d'une école (PreSkool) en suivant pas à pas les étapes illustrées ci-dessous. Vous consoliderez vos acquis en matière d'environnements virtuels, de création de projets et d'applications Django, et vous apprendrez à :

- Organiser un projet Django en plusieurs applications (faculty, student, home_auth).
- Configurer les fichiers statiques et les templates Bootstrap.
- Définir des modèles de données avec des relations (OneToOneField).
- Utiliser l'interface d'administration de Django.
- Implémenter un module d'authentification personnalisé (inscription, connexion, déconnexion).
- Lier toutes les vues, URLs et templates pour obtenir une application fonctionnelle.

À travers ce TP vous développerez une application nommée "School" comportant un tableau de bord, une gestion des étudiants avec leurs parents et un système d'authentification basé sur des rôles (étudiant, enseignant, administrateur).

Objectifs pédagogiques

À la fin de ce TP, vous devez être capable de :

- Créer un projet Django multi-applications.
- Configurer settings.py pour plusieurs applications, templates et fichiers statiques.
- Définir des modèles Django avec des relations entre eux.
- Créer et appliquer des migrations de base de données.
- Enregistrer des modèles dans l'interface d'administration Django.

- Créer un super-utilisateur et gérer les données via l'admin.
- Implémenter un système d'authentification avec un modèle utilisateur personnalisé.
- Relier les vues, URLs et templates pour un flux complet d'une application Django.

Partie 1 : Mise en place de l'environnement et du projet

Avant de commencer le développement, nous devons préparer un environnement de travail propre et isolé, puis créer la structure de base du projet Django.

1.1 Création de l'environnement virtuel

Un environnement virtuel permet d'isoler les dépendances du projet et d'éviter les conflits entre différentes versions de bibliothèques. Ouvrez un terminal dans le dossier de votre choix et exécutez la commande suivante :

```
python -m venv monenv
```

Activez ensuite l'environnement virtuel. Sur Windows :

```
monenv\Scripts\activate
```

Une fois activé, votre invite de commandes affichera le préfixe (monenv) devant le chemin courant, indiquant que vous travaillez bien dans l'environnement isolé.

1.2 Installation de Django

Django est le framework web Python que nous allons utiliser. Avec l'environnement virtuel activé, installez Django via pip :

```
pip install django
```

Pip télécharge automatiquement Django et toutes ses dépendances. Le message « Successfully installed django » confirme le succès de l'installation.

1.3 Création du projet Django

Dans Django, un projet représente l'ensemble de votre site web et contient la configuration globale. Nous allons nommer notre projet « school ». Exécutez :

```
django-admin startproject school

cd school
```

La commande django-admin startproject génère automatiquement un dossier school/ contenant le fichier manage.py et un sous-dossier school/ avec les fichiers de configuration (settings.py, urls.py, wsgi.py, asgi.py).

1.4 Création de la première application : faculty

Dans Django, un projet peut contenir plusieurs applications. Chaque application est un module indépendant qui remplit une fonction spécifique. Créons la première application nommée « faculty » :

```
py manage.py startapp faculty
```

Cette commande génère un dossier faculty/ contenant les fichiers : models.py (pour les modèles), views.py (pour les vues), admin.py (pour l'administration), apps.py et migrations/.

Partie 2 : Configuration du projet

Après la création du projet et de la première application, nous devons configurer Django pour qu'il reconnaisse notre application et sache où trouver les templates et fichiers statiques.

2.1 Enregistrement de l'application dans settings.py

Pour que Django prenne en compte votre application, vous devez l'ajouter à la liste INSTALLED_APPS dans le fichier school/settings.py. Ouvrez ce fichier et modifiez-le comme suit :

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'faculty',   # Notre application ajoutée ici
]
```

Chaque application Django doit être déclarée ici pour que le framework puisse détecter ses modèles, ses migrations et ses templates.

2.2 Création du fichier urls.py de l'application faculty

Par défaut, une nouvelle application ne possède pas de fichier urls.py. Créez un nouveau fichier nommé urls.py dans le dossier faculty/ avec le contenu suivant :

```
tp2 > school > faculty > urls.py > ...
1   from django.urls import path
2   from . import views
3
4   urlpatterns = [
5       path('', views.index, name="index")
6   ]
7
```

```
# faculty/urls.py

from django.urls import path

from . import views


urlpatterns = [

    path('', views.index, name='index'),

]
```

Ce fichier définit les routes URL spécifiques à l'application faculty. La route vide '' correspond à la page d'accueil de l'application.

2.3 Liaison avec les URLs du projet principal

Maintenant, nous devons indiquer au projet principal qu'il doit déléguer certaines requêtes à l'application faculty. Modifiez le fichier school/urls.py :

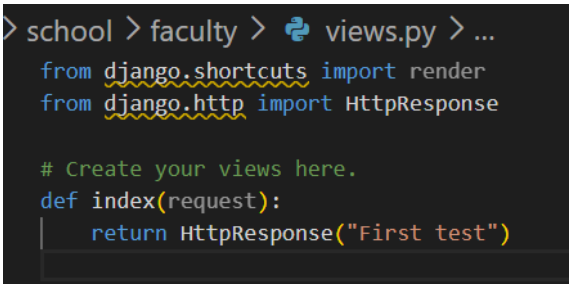
```
tp2 > school > school > urls.py > ...
1   from django.contrib import admin
2   from django.urls import path, include
3
4   urlpatterns = [
5       path('admin/', admin.site.urls),
6       path('', include('faculty.urls')),
7   ]
```

```
# school/urls.py

from django.contrib import admin

from django.urls import path, include

urlpatterns = [

    path('admin/', admin.site.urls),

    path('', include('faculty.urls')),

]
```

La fonction include() permet d'inclure les URLs d'une application dans le projet principal. Toutes les requêtes vers la racine '' seront ainsi transmises au fichier faculty/urls.py.

2.4 Création de la vue index dans faculty

La vue est une fonction Python qui reçoit une requête HTTP et retourne une réponse. Modifiez le fichier faculty/views.py pour créer une première vue simple :



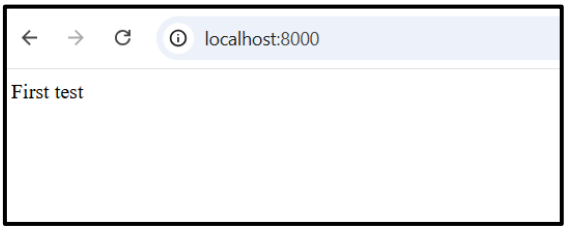

```
# faculty/views.py

from django.shortcuts import render

from django.http import HttpResponse


# Create your views here.

def index(request):

    return HttpResponse('First test')
```

2.5 Démarrage et vérification du serveur

Avant de continuer, vérifions que le projet fonctionne correctement. Lancez le serveur de développement Django :

```
py manage.py runserver
```

Ouvrez votre navigateur et accédez à l'adresse http://localhost:8000. Vous devriez voir s'afficher le texte « First test » dans la page, ce qui confirme que la vue, les URLs et la configuration sont correctement reliées.



Partie 3 : Templates et Fichiers Statiques

Pour donner un aspect professionnel à notre application, nous allons intégrer un template Bootstrap et configurer Django pour gérer les fichiers statiques (CSS, JS, images).

3.1 Création des dossiers static et templates

À la racine du projet (au même niveau que manage.py), créez deux dossiers :

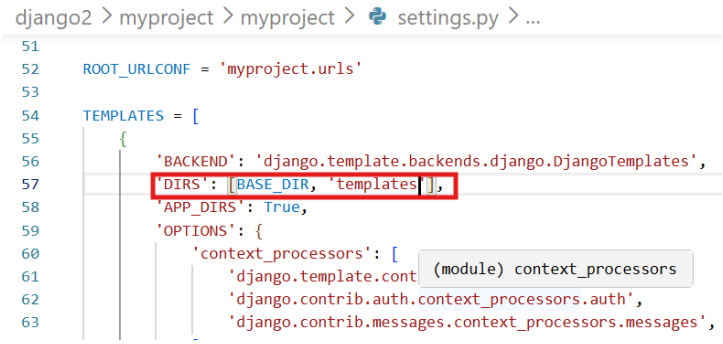
- Un dossier static/ : contiendra les fichiers CSS, JS et images.
- Un dossier templates/ : contiendra les fichiers HTML.

Voici la structure attendue :

```
school/
 ├── manage.py
 ├── school/
 ├── faculty/
 ├── static/        ← Dossier des fichiers statiques
 └── templates/     ← Dossier des templates HTML
```

3.2 Configuration de settings.py pour les templates et les statiques

Modifiez school/settings.py pour indiquer à Django où trouver les templates et les fichiers statiques. Ajoutez d'abord import os en haut du fichier, puis modifiez les sections suivantes :




```
import os  # Ajouter en haut du fichier


# Section TEMPLATES - modifier la clé 'DIRS' :

TEMPLATES = [

    {
```

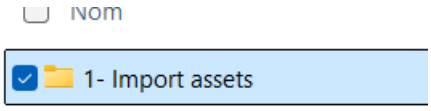
```
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [BASE_DIR, 'templates'],   # ← Modifier ici
        'APP_DIRS': True,
        ...
    },
]

# Ajouter en bas du fichier :
STATIC_URL = 'static/'
STATICFILES_DIRS = (os.path.join(BASE_DIR, 'static'),)
```

```
STATIC_URL = 'static/'
STATICFILES_DIRS = (os.path.join(BASE_DIR, 'static'),)
```

> Important
>
> Ces configurations doivent être faites dans school/settings.py (le fichier du PROJET), et non dans un fichier settings.py d'une application.

3.3 Import des assets Bootstrap

Téléchargez un template Bootstrap (le template PreSkool disponible sur le portail du cours).

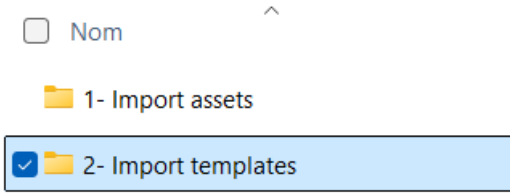


Copiez le dossier assets/ du template téléchargé dans votre dossier static/. La structure doit ressembler à :

```
static/
└── assets/
    ├── css/
    ├── js/
    └── img/
```

3.4 Intégration des templates HTML

Copiez les fichiers HTML fournis (base.html et les templates de pages) dans le dossier templates/. Pour que Django puisse générer les URLs des fichiers statiques, ajoutez la balise {% load static %} en tête de chaque template qui utilise des fichiers statiques, juste après la balise <!DOCTYPE html>. Remplacez également tous les liens vers des fichiers CSS/JS/images par la syntaxe Django :




```
{% load static %}

<!DOCTYPE html>

<html lang='fr'>

<head>

  <!-- Avant : href='assets/css/style.css' -->

  <!-- Après : -->

  <link rel='stylesheet' href="{% static 'assets/css/style.css' %}">

</head>
```

Faites de même pour tous les fichiers JS et les balises <img src='...'> dans vos templates.

3.5 Mise à jour de la vue pour utiliser un template

Modifiez faculty/views.py pour que la vue index renvoie un template HTML au lieu d'une simple réponse texte :

```
# faculty/views.py

from django.shortcuts import render

from django.http import HttpResponse


def index(request):

    # return HttpResponse('First test')  ← remplacé par :

    return render(request, 'index.html')
```

Partie 4 : Création de l'application Student, Modèles et Migrations

Dans cette partie, nous allons créer une nouvelle application Django dédiée à la gestion des étudiants, définir les modèles de données correspondants et appliquer les migrations.

4.1 Création de l'application student

Créez une nouvelle application nommée student dans votre projet :

```
py manage.py startapp student
```

4.2 Enregistrement de l'application student dans settings.py

Ajoutez 'student' dans la liste INSTALLED_APPS de school/settings.py :

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'faculty',
    'student'
]
```

```
INSTALLED_APPS = [
   'django.contrib.admin',
   'django.contrib.auth',
   'django.contrib.contenttypes',
   'django.contrib.sessions',
   'django.contrib.messages',
   'django.contrib.staticfiles',
   'faculty',
   'student',   # ← Ajouter ici
]
```

4.3 Création de urls.py pour l'application student

Créez un fichier urls.py dans le dossier student/ et liez-le au urls.py du projet principal :

```
# student/urls.py

from django.urls import path

from . import views


urlpatterns = [

    path('', views.student_list, name='student_list'),

    path('add/', views.add_student, name='add_student'),

]
```

Dans school/urls.py, ajoutez le lien vers les URLs de l'application student :

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('faculty.urls')),
    path('student/', include('student.urls')),
]
```

```
# school/urls.py

from django.contrib import admin

from django.urls import path, include


urlpatterns = [

    path('admin/', admin.site.urls),

    path('', include('faculty.urls')),

    path('student/', include('student.urls')),  # ← Ajouter

]
```

4.4 Définition des modèles dans student/models.py

Les modèles Django représentent vos données sous forme de classes Python. L'ORM (Object-Relational Mapper) de Django se charge de traduire ces classes en tables de base de données. Nous allons créer deux modèles : Parent et Student, avec une relation OneToOne entre eux.

Modèle Parent - définit les informations des parents d'élèves :

```python
class Parent(models.Model):
    father_name = models.CharField(max_length=100)
    father_occupation = models.CharField(max_length=100, blank=True)
    father_mobile = models.CharField(max_length=15)
    father_email = models.EmailField(max_length=100)
    mother_name = models.CharField(max_length=100)
    mother_occupation = models.CharField(max_length=100, blank=True)
    mother_mobile = models.CharField(max_length=15)
    mother_email = models.EmailField(max_length=100)
    present_address = models.TextField()
    permanent_address = models.TextField()

    def __str__(self):
        return f"{self.father_name} & {self.mother_name}"
```

```python
class Student(models.Model):
    first_name = models.CharField(max_length=100)
    last_name = models.CharField(max_length=100)
    student_id = models.CharField(max_length=20)
    gender = models.CharField(max_length=10, choices=[('Male', 'Male'), ('Female', 'Female')])
    date_of_birth = models.DateField()
    student_class = models.CharField(max_length=50)
    joining_date = models.DateField()
    mobile_number = models.CharField(max_length=15)
    admission_number = models.CharField(max_length=20)
    section = models.CharField(max_length=10)
    student_image = models.ImageField(upload_to='students/', blank=True)
    parent = models.OneToOneField(Parent, on_delete=models.CASCADE)

    def __str__(self):
        return f"{self.first_name} {self.last_name} ({self.student_id})"
```

```python
# student/models.py

from django.db import models


class Parent(models.Model):

    father_name = models.CharField(max_length=100)
```

```
    father_occupation = models.CharField(max_length=100, blank=True)
    father_mobile = models.CharField(max_length=15)
    father_email = models.EmailField(max_length=100)
    mother_name = models.CharField(max_length=100)
    mother_occupation = models.CharField(max_length=100, blank=True)
    mother_mobile = models.CharField(max_length=15)
    mother_email = models.EmailField(max_length=100)
    present_address = models.TextField()
    permanent_address = models.TextField()

    def __str__(self):
        return f"{self.father_name} & {self.mother_name}"
```

Modèle Student : définit les informations de l'étudiant avec un lien vers Parent :

```
class Student(models.Model):
    first_name = models.CharField(max_length=100)
    last_name = models.CharField(max_length=100)
    student_id = models.CharField(max_length=20)
    gender = models.CharField(max_length=10,
        choices=[('Male','Male'), ('Female','Female')])
    date_of_birth = models.DateField()
    student_class = models.CharField(max_length=50)
    joining_date = models.DateField()
    mobile_number = models.CharField(max_length=15)
    admission_number = models.CharField(max_length=20)
    section = models.CharField(max_length=10)
    student_image = models.ImageField(
        upload_to='students/', blank=True)
    parent = models.OneToOneField(
        Parent, on_delete=models.CASCADE)
```

```
def __str__(self):
    return f"{self.first_name} {self.last_name} ({self.student_id})"
```

Note : OneToOneField

La relation OneToOneField entre Student et Parent signifie qu'un étudiant possède exactement un Parent, et un Parent est lié à exactement un étudiant. Le paramètre on_delete=models.CASCADE indique que si le parent est supprimé, l'étudiant associé l'est également.

4.5 Installation de Pillow pour ImageField

Le champ ImageField requiert la bibliothèque Pillow pour le traitement des images. Installez-la :

```
python -m pip install Pillow
```

4.6 Création des fichiers de migration

Les migrations sont des fichiers Python générés automatiquement par Django qui décrivent les changements à apporter à la base de données. Créez les fichiers de migration :

```
py manage.py makemigrations
```

Django génère un fichier student/migrations/0001_initial.py qui crée les tables Parent et Student. Le résultat dans le terminal ressemblera à :

```
(monenv2) PS C:\wamp64\www\fstt_prof\tp2\s
 py manage.py makemigrations
Migrations for 'student':
  student\migrations\0001_initial.py
    + Create model Parent
    + Create model Student
```

4.7 Application des migrations

Appliquez les migrations pour créer effectivement les tables dans la base de données SQLite :

```
py manage.py migrate
```

Le résultat confirme l'application de toutes les migrations, y compris celles de Django lui-même (auth, admin, sessions, etc.) ainsi que celles de votre application student :

```
  Applying auth.0011_update_proxy_permissions...
 OK
  Applying auth.0012_alter_user_first_name_max_l
ength... OK
  Applying sessions.0001_initial... OK
  Applying student.0001_initial... OK
(monenv2) PS C:\wamp64\www\fstt_prof\tp2\school>
```

Partie 5 : Interface d'Administration Django

Django fournit une interface d'administration prête à l'emploi, extrêmement puissante, qui permet de gérer toutes les données de votre application sans écrire une seule ligne de code supplémentaire.

5.1 Enregistrement des modèles dans admin.py

Pour que les modèles Parent et Student apparaissent dans l'interface d'administration, vous devez les enregistrer dans le fichier student/admin.py :

```
tp2 > school > student > admin.py
1   from django.contrib import admin
2
3   from .models import Parent, Student
4
5   admin.site.register(Parent)
6   admin.site.register(Student)
```

Pour accéder à l'interface d'administration, vous devez créer un compte super-utilisateur. Exécutez la commande suivante et suivez les instructions :

```
py manage.py createsuperuser
```

Django vous demandera un nom d'utilisateur, une adresse email et un mot de passe. Si Django rejette votre mot de passe car il est trop court ou trop commun, vous pouvez ignorer l'avertissement en tapant 'y' pour forcer la création du compte.

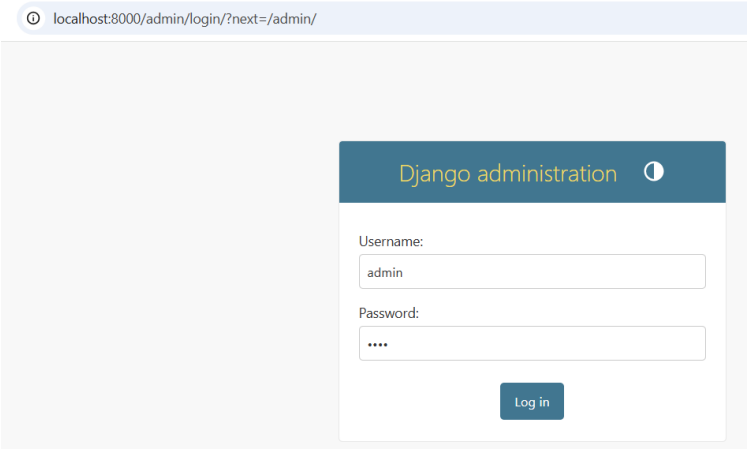
```
(monenv2) PS C:\wamp64\www\fstt_prof\tp2\school>
 py manage.py createsuperuser
Username (leave blank to use 'dell'): admin
Email address:
Password:
Password (again):
This password is too short. It must contain at l
east 8 characters.
This password is too common.
This password is entirely numeric.
Bypass password validation and create user anywa
```

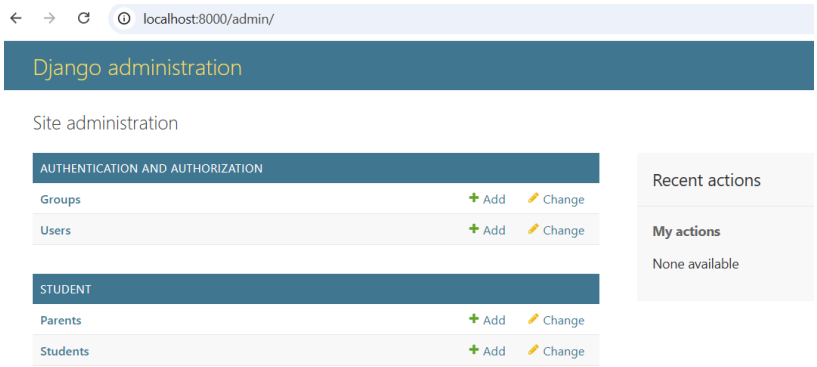
5.2 Connexion à l'interface d'administration

Lancez le serveur et accédez à l'interface d'administration :

```
py manage.py runserver

# Puis ouvrez dans votre navigateur :

# http://localhost:8000/admin/
```

Connectez-vous avec les identifiants du super-utilisateur que vous venez de créer. Vous devriez voir apparaître la section STUDENT avec les liens Parents et Students. Essayez d'ajouter un parent et un étudiant via le bouton « Add ».





5.3 Personnalisation de l'affichage dans l'admin (optionnel)

Pour personnaliser l'affichage de la liste des enregistrements, ajouter une barre de recherche et des filtres, utilisez des classes ModelAdmin :

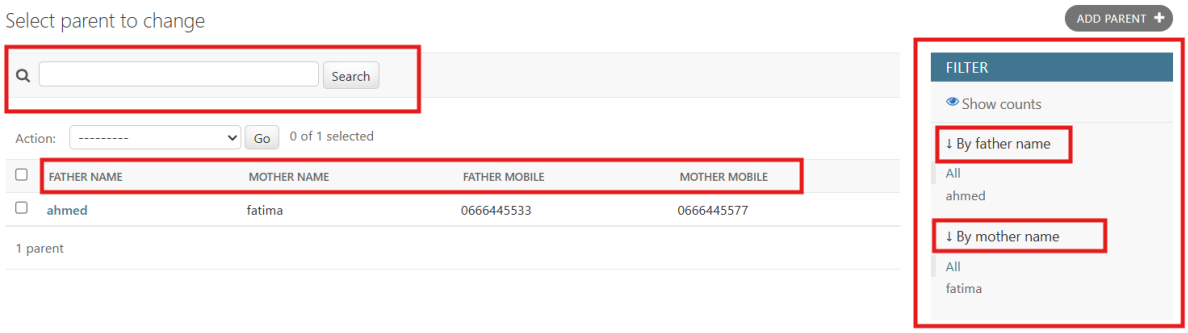
```
@admin.register(Parent)
class ParentAdmin(admin.ModelAdmin):
    list_display = ('father_name', 'mother_name', 'father_mobile', 'mother_mobile')
    search_fields = ('father_name', 'mother_name', 'father_mobile', 'mother_mobile')
    list_filter = ('father_name', 'mother_name')

@admin.register(Student)
class StudentAdmin(admin.ModelAdmin):
    list_display = ('first_name', 'last_name', 'student_id', 'gender', 'date_of_birth', 'student_class', 'joining_date',
    'mobile_number', 'admission_number', 'section')
    search_fields = ('first_name', 'last_name', 'student_id', 'student_class', 'admission_number')
    list_filter = ('gender', 'student_class', 'section')
    readonly_fields = ('student_image',)
```

```
# student/admin.py

from django.contrib import admin

from .models import Parent, Student


@admin.register(Parent)

class ParentAdmin(admin.ModelAdmin):

   list_display = ('father_name', 'mother_name',

                  'father_mobile', 'mother_mobile')

   search_fields = ('father_name', 'mother_name',

                   'father_mobile', 'mother_mobile')

   list_filter = ('father_name', 'mother_name')


@admin.register(Student)
```

```
class StudentAdmin(admin.ModelAdmin):
    list_display = ('first_name', 'last_name', 'student_id',
                    'gender', 'date_of_birth', 'student_class',
                    'joining_date', 'mobile_number',
                    'admission_number', 'section')
    search_fields = ('first_name', 'last_name', 'student_id',
                     'student_class', 'admission_number')
    list_filter = ('gender', 'student_class', 'section')
    readonly_fields = ('student_image',)
```

Avant personnalisation :

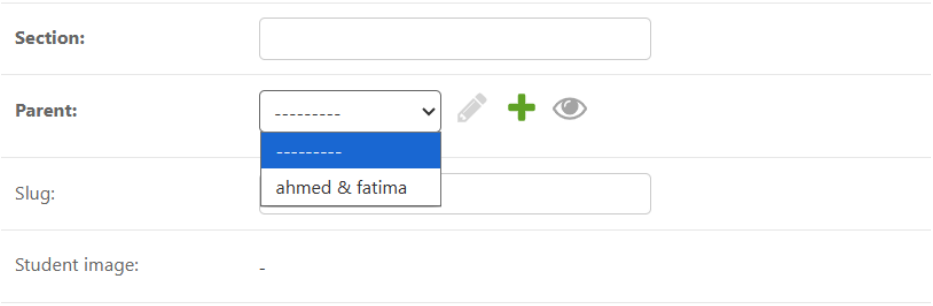
Après personnalisation :

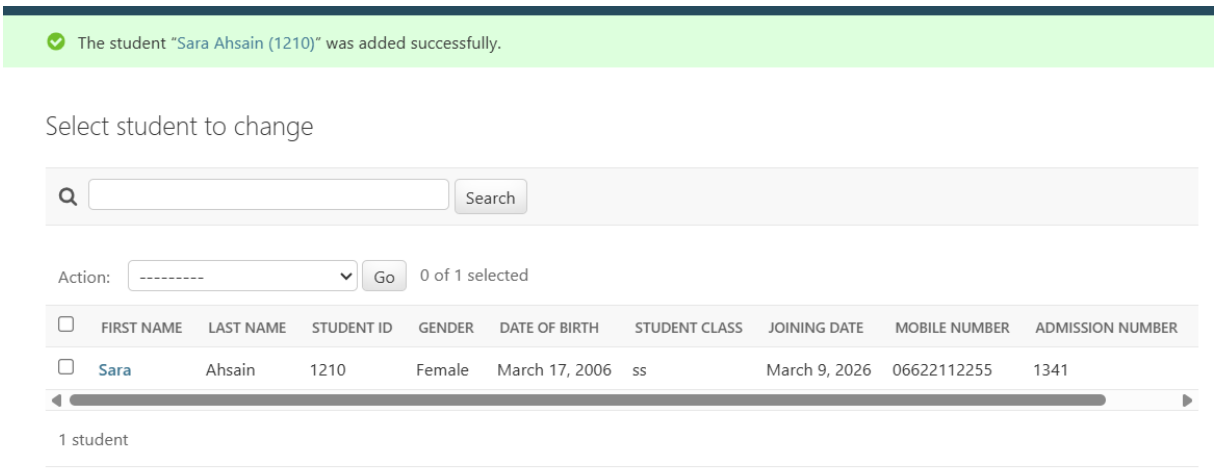


5.4 Affichage du formulaire etudiant

A travers le lien entre étudiant et parent :

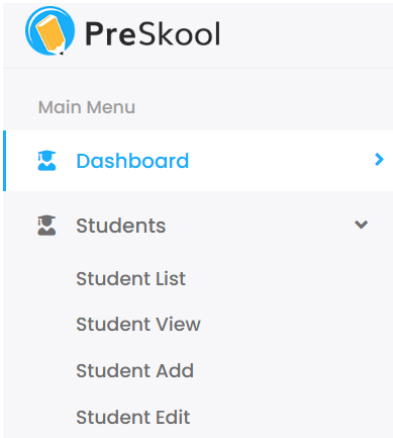
```
parent = models.OneToOneField(Parent, on_delete=models.CASCADE)
```





Partie 6 : Vues CRUD pour la Gestion des Étudiants

Maintenant que les modèles sont en place et que les données sont gérables depuis l'admin, nous allons créer des vues permettant d'effectuer les opérations CRUD (Create, Read, Update, Delete) depuis les pages web de l'application.



6.1 Mise à jour de student/views.py

Créez les fonctions de vue pour chaque opération dans student/views.py :

```python
from django.shortcuts import render, redirect

# Create your views here.
from django.shortcuts import render
from django.http import HttpResponse

def add_student(request):
    return render(request,"students/add-student.html")

def student_list(request):
    return render(request, "students/students.html")

def edit_student(request):
    return render(request, "students/edit-student.html")

def view_student(request):
    return render(request, "students/student-details.html")

def delete_student(request):
    return redirect ('student_list')
```

```python
# student/views.py

from django.shortcuts import render, redirect

from django.http import HttpResponse

from .models import Student, Parent


def student_list(request):

    return render(request, 'students/students.html')


def add_student(request):

    return render(request, 'students/add-student.html')


def edit_student(request, student_id):

    return render(request, 'students/edit-student.html')


def view_student(request, student_id):

    return render(request, 'students/student-details.html')


def delete_student(request, student_id):

    return redirect('student_list')
```

6.2 Mise à jour des routes dans student/urls.py

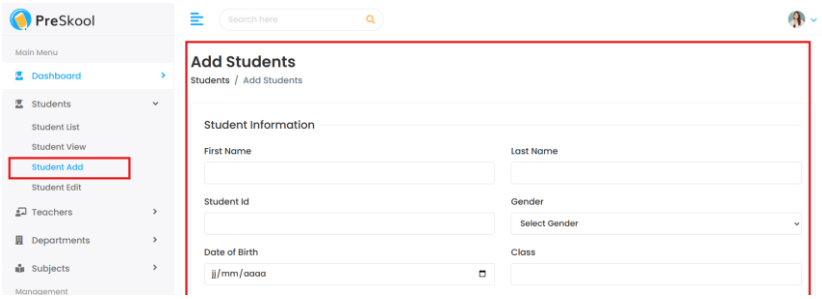
Ajoutez toutes les routes CRUD dans student/urls.py. Notez l'utilisation de <str:student_id> pour passer l'ID de l'étudiant dans l'URL :

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.student_list, name="student_list"),
    path('add/', views.add_student, name="add_student"),
]
```

```
# student/urls.py

from django.urls import path

from . import views


urlpatterns = [

   path('', views.student_list, name='student_list'),

   path('add/', views.add_student, name='add_student'),

   path('students/<str:student_id>/',

       views.view_student, name='view_student'),

   path('edit/<str:student_id>/',

       views.edit_student, name='edit_student'),

   path('delete/<str:student_id>/',

       views.delete_student, name='delete_student'),

]
```

6.3 Implémentation de la vue d'ajout d'un étudiant

La vue add_student doit traiter les données du formulaire soumis (requête POST) et créer les enregistrements Parent et Student dans la base de données.



Il est important de créer d'abord le parent, puis l'étudiant qui lui est lié :

```
def add_student(request):
    if request.method == 'POST':
        # Récupérer les données de l'étudiant
        first_name = request.POST.get('first_name')
        last_name = request.POST.get('last_name')
        student_id = request.POST.get('student_id')
        gender = request.POST.get('gender')
        date_of_birth = request.POST.get('date_of_birth')
        student_class = request.POST.get('student_class')
        joining_date = request.POST.get('joining_date')
        mobile_number = request.POST.get('mobile_number')
        admission_number = request.POST.get('admission_number')
        section = request.POST.get('section')
        student_image = request.FILES.get('student_image')

        # Récupérer les données du parent
        father_name = request.POST.get('father_name')
        father_occupation = request.POST.get('father_occupation')
        father_mobile = request.POST.get('father_mobile')
        father_email = request.POST.get('father_email')
        mother_name = request.POST.get('mother_name')
        mother_occupation = request.POST.get('mother_occupation')
        mother_mobile = request.POST.get('mother_mobile')
        mother_email = request.POST.get('mother_email')
        present_address = request.POST.get('present_address')
        permanent_address = request.POST.get('permanent_address')
```

1. Créer et sauvegarder le parent

```
        parent = Parent.objects.create(
```

```
    father_name=father_name,
    father_occupation=father_occupation,
    father_mobile=father_mobile,
    father_email=father_email,
    mother_name=mother_name,
    mother_occupation=mother_occupation,
    mother_mobile=mother_mobile,
    mother_email=mother_email,
    present_address=present_address,
    permanent_address=permanent_address
)
```

2. Créer l'étudiant lié au parent

```
student = Student.objects.create(
    first_name=first_name,
    last_name=last_name,
    student_id=student_id,
    gender=gender,
    date_of_birth=date_of_birth,
    student_class=student_class,
    joining_date=joining_date,
    mobile_number=mobile_number,
    admission_number=admission_number,
    section=section,
    student_image=student_image,
    parent=parent
)
```

3. Afficher message et rediriger vers la liste

```
messages.success(request, 'Student added Successfully')
return redirect('student_list')
```

```
else:
    return render(request, 'students/add-student.html')
```

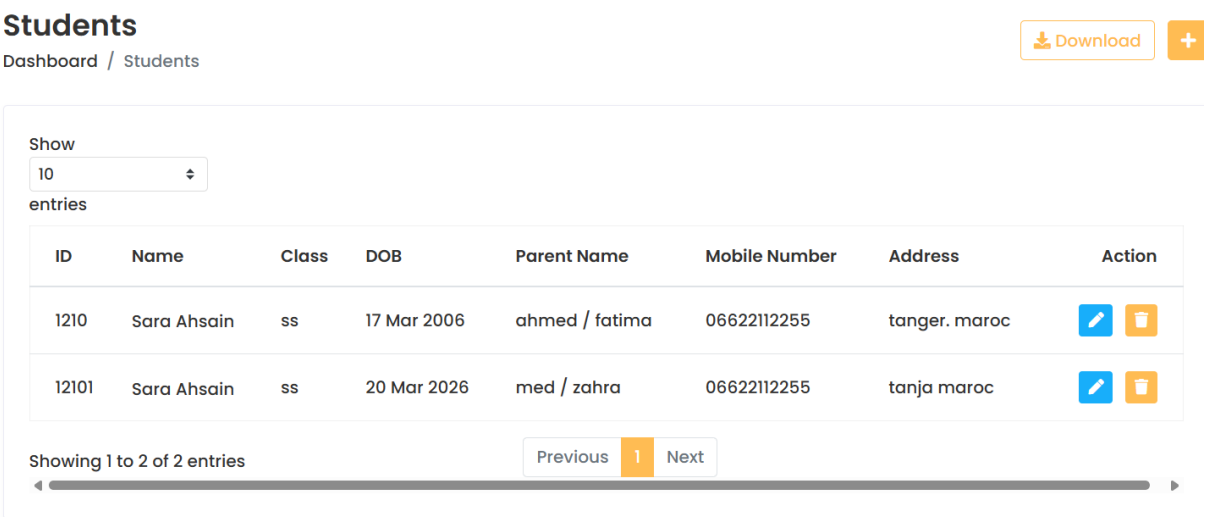
6.4 Liaison des menus aux routes

Pour relier les liens du menu de navigation de votre template base.html aux routes Django, utilisez la balise de template {% url 'nom_de_la_route' %} dans les attributs href :

```
<!-- Dans templates/Home/base.html -->
<!-- Avant : href='/student/add/' -->
<!-- Après : -->
<a href="{% url 'add_student' %}">Student Add</a>
<a href="{% url 'student_list' %}">Student List</a>
```

```python
from django.urls import path
from . import views

urlpatterns = [
    path('', views.student_list, name="student_list"),
    path('add/', views.add_student, name="add_student"),
    path('students/<str:student_id>/', views.view_student, name='view_student'),
    path('edit/<str:student_id>/', views.edit_student, name='edit_student'),
    path('delete/<str:student_id>/', views.delete_student, name='delete_student'),
]
```


```python
from django.urls import path
from . import views

urlpatterns = [
    path('', views.student_list, name="student_list"),
    path('add/', views.add_student, name="add_student"),
    path('students/<str:student_id>/', views.view_student, name='view_student'),
    path('edit/<str:student_id>/', views.edit_student, name='edit_student'),
    path('delete/<str:student_id>/', views.delete_student, name='delete_student'),
]
```



Partie 7 : Module d'Authentification personnalisé

Dans cette partie, nous allons créer un module d'authentification complet avec un modèle utilisateur personnalisé supportant plusieurs rôles : étudiant, enseignant et administrateur.

7.1 Création de l'application home_auth

Créez une nouvelle application dédiée à l'authentification :

```
py manage.py startapp home_auth
```

Ajoutez 'home_auth' dans la liste INSTALLED_APPS de school/settings.py.

7.2 Liaison des URLs dans le projet principal

Créez un fichier urls.py dans home_auth/ et liez-le au projet principal :

```
# home_auth/urls.py

from django.contrib import admin

from django.urls import path

from . import views


urlpatterns = [

    path('signup/', views.signup_view, name='signup'),

    path('login/', views.login_view, name='login'),

    path('logout/', views.logout_view, name='logout'),

]
```

Dans school/urls.py, ajoutez le lien vers les URLs de home_auth :

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('faculty.urls')),
    path('student/', include('student.urls')),
    path('authentication/', include('home_auth.urls')),
]
```

```
# school/urls.py

urlpatterns = [

   path('admin/', admin.site.urls),

   path('', include('faculty.urls')),

   path('student/', include('student.urls')),

   path('authentication/', include('home_auth.urls')),  # ← Ajouter

]
```

7.3 Définition du modèle CustomUser dans home_auth/models.py

Nous allons créer un modèle utilisateur personnalisé qui étend AbstractUser de Django et ajoute des champs de rôle :

```
# home_auth/models.py

from django.contrib.auth.models import AbstractUser

from django.db import models


class CustomUser(AbstractUser):

   is_student = models.BooleanField(default=False)

   is_teacher = models.BooleanField(default=False)

   is_admin = models.BooleanField(default=False)


   def __str__(self):

      return self.username
```

7.4 Configuration du modèle utilisateur dans settings.py

Indiquez à Django d'utiliser votre modèle CustomUser au lieu du modèle User par défaut. Ajoutez ces lignes dans school/settings.py (dans le fichier du PROJET, pas de l'application) :

```
# school/settings.py

AUTH_USER_MODEL = 'home_auth.CustomUser'

AUTHENTICATION_BACKENDS = (

    'django.contrib.auth.backends.ModelBackend',  # Backend par défaut

)

LOGIN_URL = '/authentication/login/'

LOGIN_REDIRECT_URL = '/dashboard/'
```

Important : Ordre des opérations

La variable AUTH_USER_MODEL doit être définie AVANT d'effectuer les premières migrations. Si vous l'ajoutez sur un projet existant avec des migrations, il faudra supprimer la base de données et recommencer les migrations depuis zéro.

7.5 Configuration de admin.py pour CustomUser

Enregistrez votre modèle CustomUser dans home_auth/admin.py et personnalisez les champs du formulaire de création d'utilisateur :

```
# home_auth/admin.py

from django.contrib import admin

from django.contrib.auth.admin import UserAdmin

from .models import CustomUser


@admin.register(CustomUser)

class CustomUserAdmin(UserAdmin):

    fieldsets = UserAdmin.fieldsets + (

        ('Rôles', {'fields': ('is_student', 'is_teacher', 'is_admin')}),

    )
```

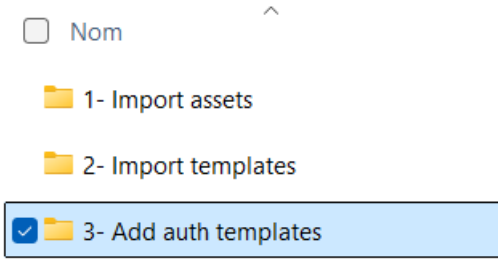
7.6 Création et application des migrations pour CustomUser

```
py manage.py makemigrations

py manage.py migrate
```

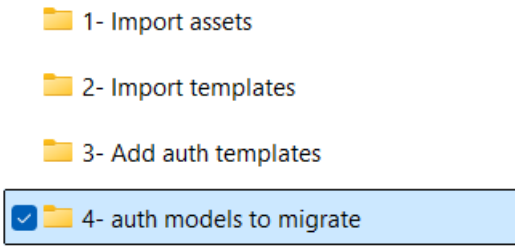
7.7 Implémentation des vues d'authentification

Créez les trois fonctions d'authentification dans home_auth/views.py : inscription (signup), connexion (login) et déconnexion (logout).

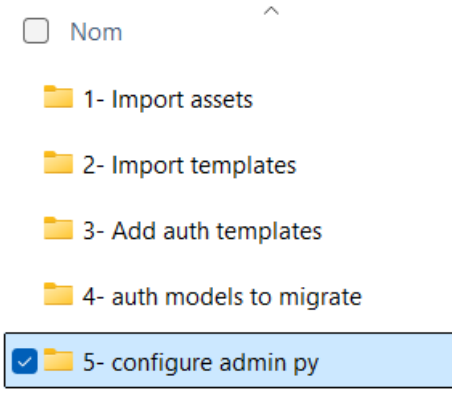
1. Premièrement, ajouter les Templates correspondantes depuis le dossier inclus avec le projet :



2. Ajouter le modèle définit sur le dossier



3. Ajouter la configuration necessaire depuis admin.py sur la nouvelle application home_auth



4. Ajouter sur settings.py du projet a la fin du fichier :

```
AUTH_USER_MODEL = 'home_auth.CustomUser'  # Adjust 'home_auth' to the app where CustomUser is defined
AUTHENTICATION_BACKENDS = (
    'django.contrib.auth.backends.ModelBackend',  # Default backend
)
```

Vue d'inscription (signup_view) :

```
# home_auth/views.py
from django.shortcuts import render, redirect
from django.contrib.auth import authenticate, login, logout
from django.contrib import messages
from .models import CustomUser

def signup_view(request):
    if request.method == 'POST':
        first_name = request.POST['first_name']
        last_name = request.POST['last_name']
        email = request.POST['email']
        password = request.POST['password']
        role = request.POST.get('role')  # student, teacher ou admin

        # Créer l'utilisateur
        user = CustomUser.objects.create_user(
            username=email,
            email=email,
            first_name=first_name,
            last_name=last_name,
            password=password,
        )

        # Assigner le rôle
```

```
        if role == 'student':
            user.is_student = True
        elif role == 'teacher':
            user.is_teacher = True
        elif role == 'admin':
            user.is_admin = True

        user.save()
        login(request, user)
        messages.success(request, 'Signup successful!')
        return redirect('index')
    return render(request, 'authentication/register.html')
```

Vue de connexion (login_view) :

```
def login_view(request):
    if request.method == 'POST':
        email = request.POST['email']
        password = request.POST['password']

        user = authenticate(request, username=email,
                            password=password)
        if user is not None:
            login(request, user)
            messages.success(request, 'Login successful!')
            # Redirection selon le rôle
            if user.is_admin:
                return redirect('admin_dashboard')
            elif user.is_teacher:
                return redirect('teacher_dashboard')
            elif user.is_student:
```

```
            return redirect('dashboard')
        else:
            messages.error(request, 'Invalid user role')
            return redirect('index')
    else:
        messages.error(request, 'Invalid credentials')
    return render(request, 'authentication/login.html')
```

Vue de déconnexion (logout_view) :

```
def logout_view(request):
    logout(request)
    messages.success(request, 'You have been logged out.')
    return redirect('index')
```

Ajouter les routes correspondantes :

```
from django.contrib import admin
from django.urls import path,include
from . views import *
urlpatterns = [
    path('signup/', signup_view, name='signup'),
    path('login/', login_view, name='login'),
    path('logout/', logout_view, name='logout')

]
```

7.8 Vue index et dashboard dans faculty/views.py

Modifiez le fichier faculty/views.py pour que la vue index redirige vers la page de login, et créez une vue dashboard accessible après authentification :

```
# faculty/views.py
from django.shortcuts import render, redirect


def index(request):
    return render(request, 'authentication/login.html')
```

```
def dashboard(request):
    return render(request, 'students/student-dashboard.html')
```

Ajoutez les routes correspondantes dans faculty/urls.py :

```
# faculty/urls.py
from django.urls import path
from . import views

urlpatterns = [
    path('', views.index, name='index'),
    path('dashboard/', views.dashboard, name='dashboard'),
]
```

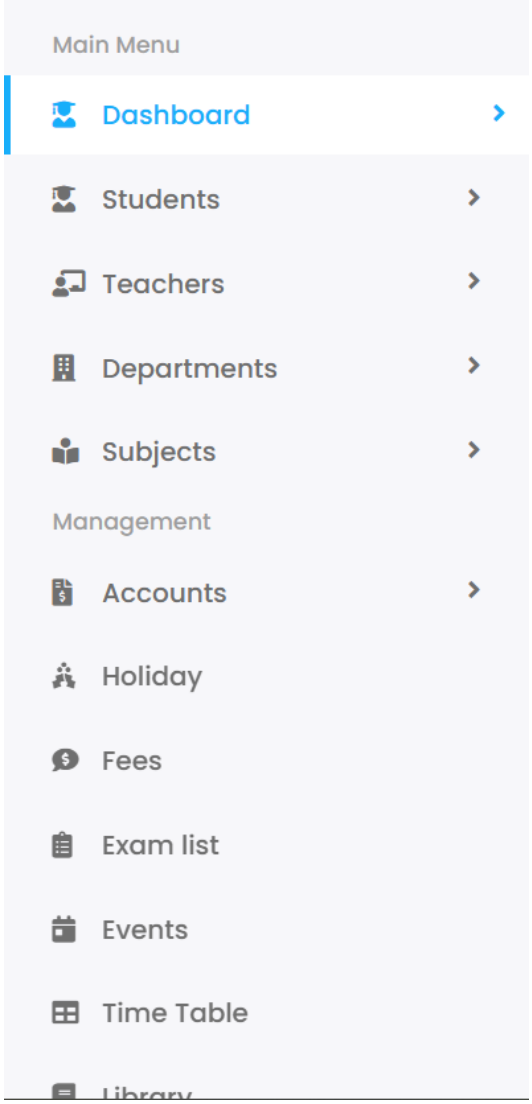
Récapitulatif des commandes

Voici un récapitulatif de toutes les commandes utilisées dans ce TP, dans l'ordre d'exécution :

```
# — Environnement virtuel ————————————————————————
python -m venv monenv
monenv\Scripts\activate

# — Installation de Django ————————————————————————
pip install django
python -m pip install Pillow

# — Création du projet ————————————————————————
django-admin startproject school
cd school

# — Création des applications ————————————————————————
py manage.py startapp faculty
py manage.py startapp student
py manage.py startapp home_auth
```

```
# — Migrations ————————————————————————————————

py manage.py makemigrations

py manage.py migrate


# — Super-utilisateur ————————————————————————

py manage.py createsuperuser


# — Lancer le serveur ————————————————————————

py manage.py runserver

# → http://localhost:8000

# → http://localhost:8000/admin/
```

Projet de fin de module : travail Demandé



Ce projet est la continuation et l'extension du TP Guide N2 portant sur la gestion des étudiants. Vous avez déjà construit l'application school avec les modules faculty, student et home_auth. Dans ce projet final, vous allez enrichir cette application pour en faire un système de gestion scolaire complet nomme PreSkool, couvrant l'ensemble des entités et fonctionnalités visibles dans le menu latéral de l'application.

L'application finale devra gérer de manière cohérente et sécurisée les modules suivants :

- Etudiants (Students) - déjà réalisé en TP2, à compléter la logique des vues pour la gestion CRUD de l'étudiant.
- Enseignants (Teachers) - gestion complète avec CRUD.
- Départements (Departments) - création et attribution des enseignants.

- Matières (Subjects) - lien avec départements et enseignants.
- Jours fériés (Holidays) - calendrier des congés de l'établissement.
- Emploi du temps (Time Table) - avec intégration optionnelle Visual Timetabling https://www.visual-timetabling.be/ .
- Examens (Exam List) - planification et saisie des résultats.
- Trois types d'utilisateurs avec des accès différencies : Admin, Enseignant, Etudiant.

La base de données utilisée peut être MySQL ou bien SQLite par défaut. Le compte rendu comprendra le code source complet déposé sur GitHub, un rapport détaillé et une démonstration vidéo des fonctionnalités principales.

Livrables attendus et critères d'évaluation

Code source - Dépôt GitHub PUBLIC

Le dépôt GitHub devra contenir :

- Le projet Django complet avec toutes les applications.
- Un fichier requirements.txt : pip freeze > requirements.txt
- Un fichier README.md contenant : instructions d'installation, identifiants de test, lien vidéo.
- Un script SQL ou fixtures Django pour initialiser la base de données avec des données de test.

Rapport technique

Le rapport devra couvrir obligatoirement les points suivants :

1. Contexte du projet : présentation de l'établissement fictif, problématique résolue.
2. Objectifs : ce que l'application accomplit.
3. Architecture générale : schéma des applications Django et leurs interactions.
4. Diagramme de classes : toutes les entités et leurs relations (UML).
5. Choix techniques : pourquoi MySQL, Django, bibliothèques tierces utilisées.
6. Description des fonctionnalités principales : une section par module.
7. Captures d'écran : au moins une capture par fonctionnalité principale.
8. Difficultés rencontrées : erreurs, problèmes de configuration, conflits.

9. Améliorations possibles : fonctionnalités non implémentées, optimisations.

Démonstration vidéo

Une courte vidéo (5 a 10 minutes) montrant :

- La connexion avec les trois types de comptes (admin, enseignant, étudiant).
- La création d'un enseignant, d'un département et d'une matière.
- La création et l'affichage d'un emploi du temps.
- La planification d'un examen et la saisie des notes.
- La consultation du calendrier des jours fériés.
- Le bonus : intégration Visual Timetabling (si réalisé).

Grille de notation

Critère d'évaluation	Points	Détails
Démonstration vidéo (fonctionnalités)	34 pts	Tous les modules fonctionnels
Qualité du code (structure, lisibilité, MVT)	20 pts	Décorateurs, séparation des responsabilités
Modèles + migrations complètes	12 pts	Modèles corrects, relations cohérentes
Système de rôles et accès sécurises	14 pts	Admin / Enseignant / Etudiant
Rapport technique complet	15 pts	Diagramme, captures, réflexion
BONUS : intégration Visual Timetabling	+5 pts	Iframe ou export JSON fonctionnel
TOTAL	95 + 5 pts	